BABEŞ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER SCIENCE
SPECIALIZATION Computer Science

DIPLOMA THESIS

Pseudo++: A Comprehensive Pseudocode Toolchain for Romanian High School Computer Science

Supervisor
Assoc. Prof. Rareș Florin Boian, PhD

Author
Bujor Mihai Alexandru

2026

Abstract

Contents

Chapter 1

Introduction

1.1 Problem Statement

1.2 Motivation

1.3 Related Work

1.4 Contributions

1.5 Thesis Structure

Chapter 2

Related Work

2.1 General-Purpose Online Code Editors

2.2 Pseudocode-Specific Tools

2.3 Educational Language Interpreters

2.4 Transpilation in Educational Contexts

2.5 Comparison with the Present Work

Chapter 3

Theoretical Background

This chapter presents the theoretical foundations underpinning the Pseudo++ toolchain. It covers the Monaco editor and its Monarch tokenisation model used for syntax highlighting, the mechanics of incremental parsing via Tree-sitter, the conceptual distinctions between interpretation and transpilation together with Pseudo++'s bifurcated pipeline, the structured-programming basis for loop equivalence transformations, the WebAssembly execution model, and the snapshot-based approach used by the reversible debugger.

3.1 Monaco Editor and Monarch Tokenisation

3.1.1 Monaco Editor

Monaco Editor [6] is the open-source code-editing component that powers Visual Studio Code. It is written in TypeScript and runs entirely in the browser, providing a full-featured editing experience — line numbers, bracket matching, glyph margins, decorations, and theming — without a server-side component.

Monaco separates two language-related concerns:

- **Tokenisation** — assigning a visual token class (keyword, string, operator, comment, ...) to each contiguous run of characters for the purpose of syntax highlighting.
- **Language services** — richer, semantic operations such as auto-completion, hover information, and diagnostics, which require a deeper understanding of the language structure.

Pseudo++ uses Monaco for its web editor, registering a custom language (`pseudocode`) and providing both a Monarch tokeniser (Section 3.1.2) and a language configuration (bracket pairs, auto-closing, comment tokens).

3.1.2 The Monarch Tokeniser Model

Monarch [7] is Monaco's declarative tokeniser specification language. A Monarch grammar is a JavaScript object describing a *finite-state transducer*: it has a set of *states*, and each state contains an ordered list of *rules* of the form $(regex, action)$.

The tokeniser processes the input line by line. In state q, it tries each rule's regex against the current position; the first match fires the associated action, which emits a token class and optionally transitions to a new state. This gives Monarch the expressive power of a *regular language* per state, with the ability to represent richer patterns (e.g. multi-line strings) through state transitions.

For the Pseudo++ language, the single `root` state suffices. Rules recognise, in priority order: comments (`#...`), string literals, numbers, the swap and assignment operators (`<->`, `<-`), comparison operators, the floor brackets (`[`, `]`), and finally identifiers — which are dispatched through case-insensitive keyword tables (`controlKeywords`, `flowKeywords`, `builtinFunctions`, `logicalOperators`) to assign the appropriate token class, falling back to `variable` for unknown identifiers.

3.1.3 Monarch vs. Tree-sitter

Monarch and Tree-sitter serve complementary but distinct roles in Pseudo++:

- **Monarch** runs entirely in JavaScript inside Monaco, is regex-based, and operates line by line. It is fast enough to re-tokenise the visible viewport on every keystroke, producing token classes consumed directly by Monaco's renderer. It has no notion of nesting, scope, or grammatical structure.
- **Tree-sitter** runs as a WebAssembly module compiled from C, performs full incremental GLR parsing, and produces a concrete syntax tree (CST) encoding the complete grammatical structure of the program. It is used exclusively by the interpreter and transpiler for execution and code generation.

The two tools are never in conflict: Monarch makes the editor *look* right; Tree-sitter makes execution *work* right.

3.2 Syntactic Analysis and Incremental Parsing

3.2.1 Classical Parsing Approaches

Given a grammar G and an input string w, a *parser* determines whether $w \in L(G)$ and, if so, constructs a parse tree witnessing the derivation. Two broad families of algorithms exist:

- **Top-down parsers** (e.g. LL(k), recursive descent) expand the start symbol top-down. They are simple to implement and produce intuitive error messages, but require the grammar to be left-recursion-free and left-factorable.
- **Bottom-up parsers** (e.g. LR(k), LALR(1)) shift tokens onto a stack and reduce them by grammar rules. They handle a strictly larger class of grammars than LL parsers and are the basis for most production parser generators (`yacc`, `bison`, `menhir`).

3.2.2 Tree-sitter and Incremental GLR Parsing

Tree-sitter [2] is an open-source parsing framework that generates Generalised LR (GLR) parsers from a grammar described in a JavaScript DSL. Its defining feature is *incremental re-parsing*: when the source text changes, the parser does not restart from scratch but instead re-uses unchanged subtrees from the previous parse tree.

Formally, let T be a concrete syntax tree (CST) for source s, and let s' be the result of applying an edit δ to s. Tree-sitter maintains a set of *valid subtrees* — nodes whose byte ranges are unaffected by δ — and constructs T' by re-parsing only the minimal affected region, grafting in the cached subtrees. The amortised complexity of an edit of size k is $O(k \log n)$ where $n = |s|$ [1]. In Pseudo++, Tree-sitter is invoked each time the user requests execution or transpilation, giving an always up-to-date CST without redundant full re-parses.

3.2.3 Concrete vs. Abstract Syntax Trees

Tree-sitter produces *concrete syntax trees* (CSTs), which contain every token including punctuation and delimiter tokens. By contrast, an *abstract syntax tree* (AST) retains only semantically meaningful nodes.

Pseudo++ works directly on the CST exposed by Tree-sitter's C API. Node types are string-tagged (`ts_node_type`), and named fields (e.g. `condition`, `var`, `start`) allow the interpreter and transpiler to locate child nodes by role rather than by index. This design avoids a separate lowering pass to an AST while retaining the benefits of a typed node API.

3.2.4 Error Recovery

Tree-sitter employs a built-in error-recovery strategy: when the parser encounters an unexpected token, it inserts `ERROR` and `MISSING` nodes into the tree rather than aborting. The resulting tree is always complete (every byte of input appears in some node) and serves as a basis for incremental syntax-error diagnostics. Pseudo++'s

linter traverses the tree looking for these sentinel nodes and reports their location to the user.

3.3 Interpretation and Transpilation

3.3.1 Interpreters

An *interpreter* is a program that directly executes a source program without producing standalone machine code. Interpreters can be categorised by the intermediate representation they operate on:

1. **Tree-walking interpreters** traverse the AST/CST and evaluate each node in turn. They are simple to implement and straightforward to extend, at the cost of performance compared to bytecode approaches.

2. **Bytecode interpreters** first compile to an intermediate bytecode (e.g. CPython's `.pyc`), then execute it in a virtual machine. This separation allows optimisations such as constant folding.

3. **JIT compilers** (e.g. LuaJIT, V8) profile execution and dynamically compile hot paths to native machine code.

Pseudo++ uses a *stack-based tree-walking interpreter*: an explicit execution stack mirrors the nesting of syntactic constructs, converting what would otherwise be recursive C calls into an iterative state machine. Each stack frame carries a *phase* counter that encodes progress through the frame's sub-steps, making the execution granularity uniform — exactly one statement per step — which is a prerequisite for the debugger's snapshot mechanism (Section 3.6).

3.3.2 Transpilation

Transpilation (source-to-source compilation) is a form of compilation where the target is another high-level language. Well-known examples include TypeScript-to-JavaScript (`tsc`), Sass-to-CSS, and Babel's ES2022-to-ES5 lowering. In the educational domain, transpilation from a didactic notation to a production language serves as a scaffolding device: students write in familiar pseudocode and observe the idiomatic equivalent in C or Pascal.

Pseudo++'s transpiler performs a single-pass tree walk over the CST, emitting target-language code for each node. A pre-pass (`transpiler_collect`) scans all assignment targets to produce the forward variable declarations required by C and Pascal, since both languages mandate that all variables are declared before use.

3.3.3 The Pseudo++ Processing Pipeline

After parsing, the CST is passed to one of two independent backends depending on the user's intent. Figure 3.1 shows the complete pipeline.

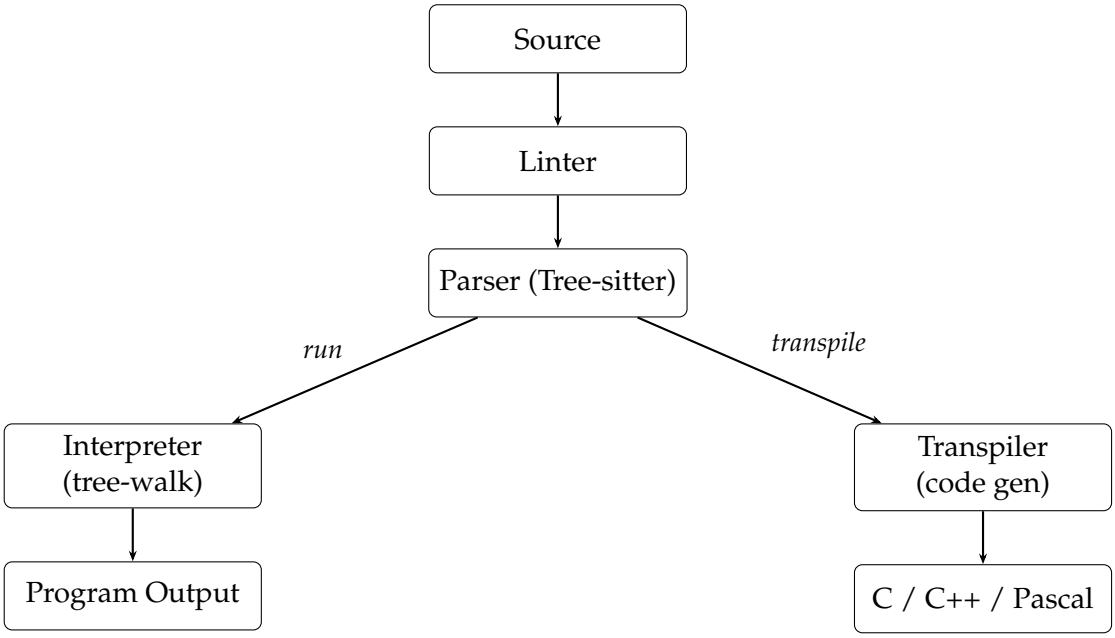

Figure 3.1: The Pseudo++ processing pipeline. After normalisation and parsing, the CST is handed to either the interpreter or the transpiler depending on the requested operation.

3.4 Equivalence of Repetitive Control Structures

3.4.1 Structured Programming and the Böhm-Jacopini Theorem

The theoretical basis for loop equivalence lies in *structured programming theory*. Böhm and Jacopini proved in 1966 [3] that any computable function can be expressed using only three control-flow primitives: sequence, selection (`if`), and a single repetition construct. A practical consequence is that the four loop forms commonly found in procedural languages — *for*, *while*, *do-while*, and *repeat-until* — are mutually interconvertible: any loop can be rewritten using any other form, at the cost of at most a guard condition or an unrolled first iteration.

This is not merely an academic curiosity. The Romanian high-school computer science curriculum [8] explicitly teaches all four loop forms and expects students to identify and apply these transformations in examination problems. Providing an automated, interactive converter therefore has direct pedagogical value.

3.4.2 Semantics of the Four Loop Forms

Let B denote the loop body (a finite sequence of statements), C a Boolean condition, i a counter variable, a the initial value, b the bound, and $s > 0$ the step size. The four loops have the following denotational semantics:

- **for**: execute B for $i = a,\ a{+}s,\ \ldots$ while $i \leq b$; zero or more iterations.
- **while**: while C, execute B; zero or more iterations.
- **do-while**: execute B, then repeat while C; at least one iteration.
- **repeat-until**: execute B, then repeat until C (i.e. while $\neg C$); at least one iteration.

The key structural distinction is between *pre-test* loops (*while*, *for*) that may execute zero times, and *post-test* loops (*do-while*, *repeat-until*) that always execute at least once. Crossing the boundary between the two families requires a guard.

3.4.3 Transformation Rules

The following semantics-preserving rewrite rules hold unconditionally:

while $\rightarrow$ do-while (and back).

$$\textbf{while}\ C\ \textbf{do}\ B \quad \equiv \quad \textbf{if}\ C\ \textbf{then}\ (\textbf{do}\ B\ \textbf{while}\ C)$$

$$\textbf{do}\ B\ \textbf{while}\ C \quad \equiv \quad B\,;\ \textbf{while}\ C\ \textbf{do}\ B$$

for $\rightarrow$ while.

$$\textbf{for}\ i \leftarrow a,\ b,\ s\ \textbf{do}\ B \quad \equiv \quad i \leftarrow a\,;\ \textbf{while}\ i \leq b\ \textbf{do}\ (B\,;\ i \leftarrow i + s)$$

repeat-until $\rightarrow$ while (and back).

$$\textbf{repeat}\ B\ \textbf{until}\ C \quad \equiv \quad B\,;\ \textbf{while}\ \neg C\ \textbf{do}\ B$$

By composing these four base rules, all twelve directed pairwise conversions among the four loop forms can be derived. Pseudo++'s `equivalence` module implements each conversion as a text-level rewrite over the CST, preserving the original source indentation.

3.5 WebAssembly

3.5.1 Overview

WebAssembly (Wasm) [5] is a binary instruction format for a stack-based virtual machine, standardised by the W3C in 2019. Its design goals are portability, safety, and near-native execution speed in the browser. A WebAssembly module is a self-contained binary (`.wasm`) that exports and imports named functions; the host (typically a JavaScript runtime) provides imports and calls exports.

Key properties of the Wasm execution model:

- **Linear memory**: a contiguous, byte-addressable memory region whose size is a multiple of 64 KiB. The C heap and stack both reside in linear memory; pointer arithmetic is valid within this region.
- **Type safety**: all instructions are typed; the validator rejects ill-typed modules before execution.
- **Sandboxing**: a Wasm module cannot access host memory outside its own linear memory or perform arbitrary I/O; all interaction with the outside world happens through explicitly imported functions.
- **Determinism**: aside from NaN bit-patterns, Wasm execution is fully deterministic.

3.5.2 Emscripten

Emscripten [9] is an LLVM-based compiler toolchain that compiles C and C++ to WebAssembly together with a JavaScript glue layer. Given a C source file, Emscripten compiles it to LLVM IR via `clang`, lowers it to Wasm via `wasm-ld` and `wasm-opt`, and generates a JavaScript companion (`.js`) that instantiates the module, provides standard C library shims, and exposes exported C functions to JavaScript callers.

Functions to be exported must be annotated with `EMSCRIPTEN_KEEPALIVE` to prevent dead-code elimination, or listed via the `-sEXPORTED_FUNCTIONS` linker flag.

3.5.3 Application in Pseudo++

The Pseudo++ interpreter is written in C23 and compiled to Wasm via Emscripten. A thin bridge layer (`wasm/bridge.c`) exposes the interpreter lifecycle and I/O routines:

```
EMSCRIPTEN_KEEPALIVE runtime_t* runtime_create_buffered(void);
EMSCRIPTEN_KEEPALIVE exec_state_t runtime_step(runtime_t* rt);
EMSCRIPTEN_KEEPALIVE const char* runtime_pop_output(runtime_t* rt);
EMSCRIPTEN_KEEPALIVE void runtime_push_input(runtime_t* rt,
                                             const char* line);
```

The I/O abstraction layer (`io_buffered`) replaces the standard `stdio` calls with a queue-based mechanism: output is pushed into a string queue that JavaScript drains via `runtime_pop_output`, and input is delivered by JavaScript calling `runtime_push_in` before the interpreter requests it. This non-blocking design ensures the browser's event loop is never stalled by a long-running computation.

3.6 Statement-Level Step-Through Debugging

3.6.1 Interactive Debugging in Educational Contexts

Step-through debugging allows a student to observe a program's behaviour one statement at a time, inspecting variable values and control flow at each point. This is particularly valuable when learning programming: rather than reading static code, the student sees the program *in motion* — which variable changes, which branch is taken, how a loop counter evolves.

Pseudo++'s debugger exposes two user-facing operations:

- **Step Into** (one statement at a time): executes the next statement, highlights the corresponding source line in the editor, and updates the variable inspection panel.
- **Continue** (run to completion): resumes fast execution without step-by-step visibility, stopping only when the program finishes, encounters a runtime error, or needs user input.

3.6.2 Per-Step State Capture

At each Step Into, the runtime first serialises the complete program state into a `runtime_snapshot_t` structure before advancing execution. A snapshot records:

- the variable environment: name, value, and type for every variable currently in scope;
- the execution stack: frame type, phase counter, loop bounds, loop counter, loop variable name, and the identity of the current CST node for each active frame;

- auxiliary I/O state: the multi-variable read index and a pending-input flag;
- the current line number and execution-state enum.

The snapshot identifier is stored by the JavaScript `PseudoDebugger` class alongside the cached variable values and condition-evaluation metadata for that step. Variable values are cached on the JavaScript side so that the variable panel can be refreshed without an additional round-trip to the Wasm module.

3.6.3 Condition Visualisation

When the statement about to execute is a conditional or loop guard, the runtime additionally serialises the condition expression text and its boolean result into a small JSON object. The front-end displays this beneath the variable panel, letting the student see not only *which* branch was taken but *why* — the exact condition text and its evaluated truth value at that step.

Chapter 4

The Pseudo++ Toolchain

4.1 System Architecture

4.2 Language Grammar

4.3 Source Normalization

4.4 Parsing and AST Construction

4.5 Interpreter

4.5.1 Value Representation

4.5.2 Execution Environment

4.5.3 Expression Evaluation

4.5.4 Control Flow

4.6 Debugger

4.7 Transpiler

4.7.1 Code Generation for C, C++, and Pascal

4.7.2 Loop Equivalence Transformations

4.8 I/O Abstraction and WebAssembly Integration

4.9 Web Editor

4.10 VSCode Extension

Chapter 5

Evaluation

5.1 Testing Methodology

5.2 Unit Testing

5.3 Integration Testing on BAC Examination Subjects (2020–2025)

5.3.1 Dataset Description

5.3.2 Results

5.4 Transpiler Correctness Verification

5.5 Performance Analysis

5.6 Discussion

Chapter 6

Conclusions

6.1 Summary

6.2 Limitations

6.3 Future Work

Bibliography

[1] Max Brunsfeld. An incremental parsing system for programming tools. Master's thesis, Massachusetts Institute of Technology, 2018.

[2] Max Brunsfeld et al. Tree-sitter: An incremental parsing system for programming tools. In *Proceedings of the USENIX Workshop on Hot Topics in Operating Systems (HotOS)*, 2018.

[3] Corrado Böhm and Giuseppe Jacopini. Flow diagrams, turing machines and languages with only two formation rules. *Communications of the ACM*, 9(5):366–371, 1966.

[4] Jakob Engblom. A review of reverse debugging. In *Proceedings of the 2012 System, Software, SoC and Silicon Debug Conference (S4D)*, pages 1–6, 2012.

[5] Andreas Haas, Andreas Rossberg, Derek Schuff, Ben Titzer, Dan Gohman, Luke Wagner, Alon Zakai, JF Bastien, and Michael Holman. WebAssembly specification. Technical report, World Wide Web Consortium (W3C), 2019.

[6] Microsoft Corporation. Monaco editor. `https://microsoft.github.io/monaco-editor/`, 2016.

[7] Microsoft Corporation. Monarch: Create declarative language syntax highlighters. `https://microsoft.github.io/monaco-editor/monarch.html`, 2016.

[8] Ministerul Educaţiei Naţionale. Programa şcolară pentru disciplina informatică, clasele IX–XII. Technical report, Ministerul Educaţiei Naţionale al României, 2004.

[9] Alon Zakai. Emscripten: An LLVM-to-JavaScript compiler. In *Proceedings of the ACM International Conference Companion on Object Oriented Programming Systems Languages and Applications (OOPSLA)*, pages 301–312, 2011.